Question 1

a) One of the roles of an Operating System (OS) is that of an illusionist. Briefly describe two illusions that an OS provides.

[4 marks]

	Do not write in this column
Two from a selection of possible	
1. Processes (the CPU is doing many things at once), [2 marks],	
2. Memory (my program can use all the memory), [2 marks].	
	4 marks

b) Consider the following processes and the CPU burst times they need in order to complete.
P1= 5ms P2= 4ms P3= 6ms P4= 5ms Assume no process interrupts for I/O and that processes are in the queue in the above order at time t=0.

i) Calculate the average wait time for the First Come First Served (FCFS. [4 marks]

ii) Calculate the average wait time for Round Robin (RR with q=2ms) scheduling algorithms [4 marks]

[8 marks]

	Do not write in this column
i)For FCFS	
Wait: (respectively): 0, 5, 9, 15, so avg. wait: 29/4 = 7.25ms [4 marks].	
ii)For RR	
Wait: (respectively): 0+6+6, 2+6, 4+6+3, 6+6+3 = 48, so avg. wait = 12ms [4 marks].	
	8 marks

c) Answer the following:
i) What purpose does the dual mode operation (kernel-user) serve? [2 marks]
ii) Briefly describe 3 ways in which a switch from user to kernel mode can occur. [6 marks]

[8 marks]

	Do not write in this column
i) It is used for protecting the OS from user processes. [2 marks].	
ii) Trap, system call, interrupt. Short description for each, e.g. for system call a user process requests a kernel service, control is passed to kernel via secure mechanism. Trap is a like a s/w interrupt and can be triggered by e.g. protection violation, divide by zero, etc [6 marks (2 for each)].	
	8 marks

d) Answer the following questions about Process Control
i) Can a process create another process? [1 mark]
ii) Define the methods contain in os module in python that can call POSIX system calls [4 marks]

[5 marks]

	Do not write in this column
i) Yes, unique identity of each process is the process ID (PID) [1 mark].	
ii) os.fork() – system call to create a copy of the current process, and start it running – os.execv (various versions) – system calls to change the program being run by the current process – os.waitpid() – system call to wait for a process to finish – signal.signal() – system call to send a notification to another process (from signal module) [4 marks].	
	5 marks

Question marking:

$$\frac{\quad}{4} + \frac{\quad}{8} + \frac{\quad}{8} + \frac{\quad}{5} = \frac{\quad}{25}$$

Question 2

a) What are the four main functions of a File System?

[4 marks]

	Do not write in this column
- Disk Management: collecting disk blocks into files [1 mark] – Naming: Interface to find files by name, not by blocks [1 mark] – Protection: Layers to keep data secure [1 mark] – Reliability/Durability: Keeping of files durable despite crashes, media failures, attacks, etc [1 mark]	
	4 marks

b) Answer the following questions on file permissions

i) As a file administrator you are about to assign rights to a file 'test.txt' to different level of users. What command you will write in Linux to assign all three rights (RWX) to everyone (u,g,o) [1 mark]

ii) Clearly differentiate the two types of file links [4 marks]

[5 marks]

	Do not write in this column
i) chmod 777 test.txt [1 mark]	
ii) Two types of links: – Hard links (ln original link) • Linked files share the same inode number • Hard-linking of directories is not allowed [2 mark] – Symbolic (soft) links (ln –s original link) • File contains (full path) name of the original file • Similar to Windows shortcut [2 mark]	
	5 marks

c) Assume that the disk head is on track 80 and that at that moment the queue of requests for accessing tracks is as follows: [95, 60, 8, 135, 21, 41]. Show how a Shortest Seek Time First (SSTF) disk scheduler would process the queue and use this scenario to illustrate SSTF's main disadvantage. [5 marks]

[8 marks]

	Do not write in this column
The queue would be processed in this order for SSTF: 95, 60, 41, 21, 8, 135. [6 mark] The danger is that as requests dynamically arrive in the queue, if they are clustered close to 8 before 135 gets served, 135 may wait for a very long time (or forever) – unbounded wait that can lead to starvation. [2 mark]	
	8 marks

d)) Consider the case of I/O using Direct Memory Access (DMA). Answer the following:
i) Briefly describe how DMA works. [3 marks]
ii) What is the main benefit of using DMA for I/O? [2 marks]

[5 marks]

	Do not write in this column
i)Data transfer between memory and device without the CPU – The hardware must have a DMA controller for this to happen, typically on the mainboard _The DMA controller has access to memory bus independent of the CPU _The CPU is interrupted only at start and end of the transaction – rather than for each memory read / write if the CPU was doing the job itself [3 mark]	
ii) Without DMA the CPU would typically be occupied only with the read/write operation for the whole duration (Programmed I/O) [2 mark]	
	5 marks

e) Provide any three advantages of SSDs over HDDs

(3 marks)

	Do not write in this column
1. SSDs can be seen as more reliable due to the lack of mechanical moving part	
2. For random-access I/O, SSDs offer a big advantage. For HDDs head movement would cause significant delays leading to decaying performance. SSDs do not suffer from this	
3. SSDs also very light weight, low power, silent	
	3 marks

Question marking: $\frac{\quad}{4} + \frac{\quad}{5} + \frac{\quad}{8} + \frac{\quad}{5} + \frac{\quad}{3} = \frac{\quad}{25}$

Question 3

a) What kind of hardware support is needed for paged virtual memory to work?

[4 marks]

	Do not write in this column
There needs to be an MMU [0.5 marks] that can remap virtual pages to physical	
pages [1.5 marks]. Also, when a page not currently mapped is referenced [1 mark],	
there needs to be a trap to the OS so it can fetch the page [1 mark].	
	4 marks

b) A computer with a 32-bit address uses a 2-level page table. Virtual addresses are split into a 9-bit "level 1" page table field, an 11-bit "level 2" page table field, and an offset.

What is the size of the pages and how many are there in the address space? Show all your work.

[3 marks]

	Do not write in this column
Twenty bits are used for the virtual page numbers [1 mark], leaving 12 bits for the	
offset [0.5 marks]; this results in a 2^{12} = 4096 bytes page size [1 mark]. Twenty bits	
for the virtual page leads to a total of 2^{20} pages [0.5 marks].	
	3 marks

c) Fill in the SIX (6) gaps in the text below, which refers to paging and address translation. You are only required to write down what the blanks marked **A** to **F** corresponding to:

"During address translation, we translate _____**A**_____ addresses to _____**B**_____ addresses.

Logical addresses consist of page number and _____**C**_____. The _____**D**_____ bits do not change during translation; we just translate the _____**E**_____ number to a _____**F**_____ number."

Note: Each gap corresponds to a single word.

[3 marks]

	Do not write in this column
A. logical **D.** offset	
B. physical **E.** page	
C. offset **F.** frame	
[6*0.5 marks: each of the 6 correctly identified gaps]	
Note: Other wording may also be accepted, if in context and similar in meaning.	
	3 marks

d) Consider the statement: "The locality of reference has impact on replacement policies as 'victim' pages can be used on the assumption of e.g., 'Least Recently Used'."

Indicate whether you agree (or not) with this statement and justify your answer.

[3 marks]

	Do not write in this column
I agree with this statement [0.5 marks].	
If a page has not been used for a long time because of temporal locality, we do not	
expect it to be used again soon [1.5 marks] and we therefore consider it as a likely	
"victim" for eviction from main memory [1 mark].	
	3 marks

e) Consider a system that uses 8-bit addressing and pages that have a size of 16 bytes. A process **P1** has 4 logical pages. Error! Reference source not found. below shows the logical page to physical frame mapping for process **P1**.
Process **P1** cannot use more than 3 physical frames (e.g., because there is no more space in physical memory).
You can assume that the execution of **P1** is not interrupted by another process.

Logical Page number	Frame number
0	1
1	2
2	0
3	-

Table 1

When a page fault occurs, the paging system uses the Least Recently Used (LRU) policy to evict a page. An associative TLB is also used that can store only 2 entries. The TLB also uses the LRU policy to update its contents when full.
Assume that **P1** accesses the following logical addresses in order: **12**, **16**, **41**, **17**, **42**, **60**.

[12 marks]

Answer the following questions, explaining your work:

i) Which of the logical addresses will result in a TLB miss and which will result in a TLB hit? You can assume that the TLB is empty before the accesses start.

(5 marks)

ii) Which of the logical addresses (if any) will result in a page fault?

(3 marks)

iii) After accessing the logical address **60**, which physical address is eventually accessed by the system (after servicing any page faults that may arise)? Show any calculations needed. Present your result in decimal.

(4 marks)

	Do not write in this column
i)	**5 marks**
For 8-bit addressing with 16-byte pages, we need 4 bits for the offset so that leaves	
4 bits to number pages [1 mark].	
Since logical address **12** is in page **0** (logical addresses **0-15**); **16** in page **1** (logical	
addresses **16-31**); **41** in page **2** (logical addresses **32-47**); **17** in page **1**; **42** in page **2**	
in page **2**; **60** in page **3** (logical addresses **48-73**), then the page access sequence is:	
0, 1, 2, 1, 2, 3 [2 marks].	
The TLB has only 2 entries, so a miss will be created by **0** (miss), **1** (miss), **2**	
(miss, replaces **0**), **1** (hit), **2** (hit), **3** (miss, replaces **1**) [2 marks].	

	Do not write in this column
ii)	**3 marks**
The only logical address that will cause a page fault is **60**, because it maps to page **3**	
(for which there is no corresponding frame) [2 marks]. The page that will be evicted	
is the LRU page **0** (see also the answer to *part i)*) [1 mark].	
iii)	**4 marks**
Page **3** will be mapped to frame **1**, taken from the evicted page **0** (see also the answer	
to *part i)*) [1 mark].	
Logical address **60** had offset **12** inside page **3** (as this page starts at logical address	
48) [1 mark], so the offset will remain the same in frame **1** [1 mark].	
Therefore, the physical address will be **28** = 16 + 12 [1 mark].	
	12 marks

Question marking: $\frac{\quad}{4} + \frac{\quad}{3} + \frac{\quad}{3} + \frac{\quad}{3} + \frac{\quad}{12} = \frac{\quad}{25}$

Question 4

a) Answer the following questions about threads:

[11 marks]

i) What is "co-operative thread termination" and why is it a good idea to use it, instead of non-cooperative (i.e., forced) termination?

(6 marks)

ii) What is the main benefit of supporting multiple kernel-threads per process?

(5 marks)

	Do not write in this column
i)	**6 marks**
Co-operative termination means that a thread can be asked to terminate [1 mark].	
This approach allows threads to release any locks [1 mark], having left critical	
regions [1 mark] (i.e., restored any data structures to a consistent state [0.5 marks]).	
Non-cooperative (i.e., forced) termination would either risk ending a thread holding a	
lock [1 mark] or risk allowing a new thread to enter a critical region that had not been	
completed [1.5 marks].	
ii)	**5 marks**
Kernel-threads are scheduled by the OS scheduler [1 mark] to one of the CPU cores	
[1 mark]; many such threads in a process allow a single user process to use multiple	
CPU cores [1.5 marks]. This means we can fully exploit the hardware's power	
without the overheads of multiple processes [1.5 marks].	
	11 marks

b) A **Mailbox** class is to be used by multiple threads; its intended behaviour is:

- When the mailbox is written, it becomes occupied. When the contents are read, the mailbox becomes empty.
- The **write** method is used to place something in the mailbox; it should block if the mailbox is already occupied.
- The **read** method is used to extract something from the mailbox; it should block if the mailbox is empty.

```
public class Mailbox <T> {
  private T contents;
  public Mailbox() {
    contents = null;
  }
  public void write(T entry) {
    contents = entry;
  }
  public T read() {
    T temp = contents;
    contents = null;
    return temp;
  }
}
```

Figure 1

Consider the code above in **Figure 1**; it compiles but is incorrect. Then answer the questions:

[8 marks]

i) Explain how the absence of locking could cause the code to misbehave.

(3 marks)

ii) Describe the changes needed to correct the code. **Note**: You are not required to write code.

(5 marks)

	Do not write in this column
i)	**3 marks**
Code can misbehave because it allows the two threads to make changes at the same	
time [1 mark]. Therefore, a full mailbox may also be overwritten [1 mark] and an	
empty mailbox may be read [1 mark].	
ii)	**5 marks**
The required changes are:	
1. Synchronize the **read()** and **write()** methods [1.5 marks].	
2. Wait for the mailbox to be empty (before writing) [1 mark] or full (before	

reading), using the **wait()** method [1 mark].	

	Do not write in this column
3. Notify other waiting threads of changes to the state [1.5 marks].	
4. Throw an interrupted exception [1 mark] when a waiting thread is woken by a	
termination request [1 mark].	
	8 marks

c) Explain whether the arrangement of threads and resources shown below can lead to deadlock, or not:

- Thread **T1** will try to get resources **R1** and **R2** in a random order.
- Thread **T2** will try to get resources **R1** and **R2** in a fixed order: first **R1**, then **R2**.

Assume that: **a)** all resources have only one instance and **b)** threads run concurrently. You can use a resource allocation graph in your answer, but it is not necessary. If you think that the arrangement shows a deadlock, suggest how deadlock can be avoided.

[6 marks]

	Do not write in this column
There is deadlock [0.5 marks], as we can have a cycle if **T1** goes for **R2** and then **R1**	
[1 mark], represented as T1 → R2 → T2 → R1 →T1 [2.5 marks].	
We can avoid deadlock [0.5 marks] if we enforce a strict ordering in **T1** [1 mark]	
(e.g., go for **R1** and then **R2** [0.5 marks]).	
	6 marks

Question marking: ___ + ___ + ___ = ___
11 8 6 25